

CSCI 3334 Final Project Report

The goal of this project is to build a concurrent task dispatcher in Rust. The system will simulate a stream of incoming work, place that work into one or more queues, and assign tasks to a bounded worker pool according to a scheduling policy. This project is not only about “getting code to run.” It is about showing that we understand how systems are built from smaller parts: ownership, threads, channels, shared state, locks, worker pools, queues, fairness, throughput, and shutdown.

The threads and major components in the design are a small set of concurrent components. Each of them has a singular role. The generator thread, `generator_loop` in the code, creates exactly 1000 tasks, randomizes the task order, and emits them over time. In 20ms baseline or the burst behavior for the stress workload. Next is the dispatcher thread, `dispatcher_loop` in code. This thread manages the queues and would be considered the brain of the system. It receives tasks, tracks available workers, enforces global CPU cap, which is 100%, and decides which task to send to workers according to the scheduling policy. Next is the worker pool, which is the 8 worker threads and `worker_loop` in the code. Each worker waits for a command, runs a task and reports completion back to the dispatcher. Next is the monitor thread, `monitor_loop` in the code. This thread samples system states every 10ms, CPU reserved and number of active workers, logs to CSV, and sends samples to the collector. Next is the collector thread, `collector_loop` in the code. This thread aggregates task completion events and monitor samples, then prints the final metrics. These metrics are the total runtime and supplementary metrics. Finally, is the main thread or `main` in the code. It takes the configurations and does 4 runs. These are both experiments, balanced workload and stressed workload, in FIFO and

Optimized scheduling. All in all, every thread works together to receive the final output, which takes a bit of time to fully process.

The data that is shared and how it is protected? Most of the data is not shared directly. Instead, it is owned by one thread and communicated through channels. Shared data is limited and protected using atomics. Data that is owned by the dispatcher is the task queues and worker availability queues. These are safe because they are only accessed inside the dispatcher thread. The data that is shared is `cpu_in_use` which is the current reserve CPU percentage. Next is `active_workers` which is how many workers are currently running tabs. Then `stop_flag` which is the signal for monitor thread to stop sampling. Why atomic works here though is because the values are small counters and flags. They only require simple reads and writes.

Channels were used to keep the design modular and avoid locking shared structures. The first was for the generator and workers communicating with the dispatcher. This was the Event Channel. Why it was used was because the dispatcher is the single owner of the scheduling logic. So, the generator and workers send “events” through the channel to the dispatcher. That way the dispatcher has a unified inbox for system events. The next channel was for the dispatcher communicating with the workers. For this, it was the per-worker WorkerCmd channels. Why it was used was because workers are independent and block waiting for a task. This channel makes it easy to address a specific worker and keep worker logic simple. The final channel was for the dispatcher and monitor communicating with the collector. Why it was used was because the collector aggregates results and prints metrics. This separates system execution from stats and reports.

Shared state was used in a few places in the code. It was used in `cpu_in_use`, it is shared so the monitor can read current reserved CPU without asking the dispatcher. Without it, it would

add extra messages and overhead. It was used in `active_workers` so that the monitor can sample how many workers are active without coordinating with each worker. Then `stop_flag` to allow a clean shutdown of the monitor loop when the run completes. In the end, this keeps the monitor independent and avoids interfering with the dispatcher scheduling.

I implemented two scheduling policies under a global CPU cap. These were Strict FIFO and Optimized Fit. For Strict FIFO, all tasks go into one FIFO queue, and the dispatcher only considers the head task. If the head task would exceed the CPU cap, dispatch stalls even if later task could fit. For Optimized Fit, tasks are separated into IO queue and CPU queue. Dispatcher chooses a task that fits under the CPU cap. So, when the CPU is tight, `cpu_in_use >= 80`, then prefer IO tasks. If anything, prefer CPU first then IO as fallback. This helps reduce and minimize total runtime, especially under burst CPU heavy workloads.

The behavior that was improved because of the policy was the total runtime. The optimized policy improves total runtime and typically improves system throughput. Especially in the stressed workload. Under CPU cap, FIFO can block when the head task doesn't fit. However, optimized policy can often schedule an IO task instead, keeping workers active. Burst and long CPU tasks create backlog, optimized policy keeps fitting tasks flowing. More time with workers doing useful work instead of waiting for CPU to free. The behavior that became worse or more complicated because of the policy was fairness and simplicity. If CPU is tight frequently, the dispatcher tends to prefer IO tasks which can cause CPU tasks to wait longer. Optimized requires multiple queues and since it is heuristic-based, result may vary with burst patterns and random sees more than FIFO. So, while Optimized does improve total runtime at the cost of fairness and simplicity.

A concurrency bug or design mistake I did hit during development was passing `cfg` into multiple threads moving it into one closure, making it unavailable. Another was the shutdown ordering. The monitor stopped too early, or the collector exited before receiving final messages so metrics would be incomplete. I fixed the `cfg` problem by coping and cloning `SimConFlg` and `Policy` so each thread can receive its own copy. For the collector problem, I made the collector wait until it received both `SimulationDone` and `MonitorDone`. This then allowed all samples and results to be included. There were many more problems, but these were the major ones.

Starvation and Unfairness is still possible for both policies. For FIFO, a CPU-heavy task at the front can block the entire queue when CPU cap is tight, even if many IO tasks behind it fit. This causes unfair waiting for tasks behind the head. As mentioned earlier, Optimized has unfairness when the CPU is tight and it prefers IO tasks. However, if IO tasks keep coming, especially in burst, the CPU tasks may wait longer.

Finally, talk about metrics. I made the metrics go to 4 different CSV files to separate them since every time a new run was completed, it would overwrite the CSV file. The primary metric is the total runtime, and it measures the clock time from the start of the run to the moment all tasks are completed and the system cleanly shuts down. It would compute by `sim_end - sim_start` in the collector. Experiment A showed that Optimized had an improved runtime compared to FIFO due to its better CPU usage, fewer dead zones and more workers active. Experiment B showed that both policies were around the same total runtime with less than a 100ms difference. The main lesson I learned was that runtime optimization is mostly about avoiding idle time and stress testing changes the metric heavily.